

Text Mining Project

Master in Data Science and Advanced Analytics

NOVA Information Management School

Universidade Nova de Lisboa

Market Sentiment Classification of Financial Tweets

Group 24

Pedro Santos, 20250399

Tiago Duarte, 20250360

Spring Semester 2025–2026

Contents

1	Introduction	1
2	Data Exploration	2
2.1	Dataset Overview and Class Distribution	2
2.2	Tweet Length Distribution	2
2.3	Vocabulary and Word Clouds	2
2.4	Linguistic and Domain-Specific Observations	2
2.5	Key Findings	3
3	Data Preprocessing	4
3.1	Corpus Split	4
3.2	Text Cleaning and Normalisation	4
3.3	Tokenisation and Stop Word Removal	4
3.4	Lemmatisation and Stemming	5
4	Feature Engineering	5
4.1	Bag-of-Words and TF-IDF	5
4.2	Word Embeddings: Word2Vec and GloVe	5
4.3	Transformer Encoder Embeddings	6
4.4	Extra Work – Additional Transformer Encoder Methods	6
5	Classification Models	7
5.1	Traditional and Deep ML Models	7
5.2	Frozen Encoder Embeddings as Features	7
5.3	Fine-tuned Transformer Encoders	8
5.4	Extra Work – Decoder Model for Classification	8
6	Evaluation and Results	9
6.1	Overall Model Comparison	9
6.2	Per-Class Analysis	10
6.3	Final Model Selection	10
6.4	Extra Work – Agentic AI Workflow (Extra Challenge 2)	10
7	Conclusions	12
	References	13
A	EDA Material	i
B	Preprocessing Pipeline Summary	iii
C	Decoder Model Prompts	vi
D	Model Results	vii
E	Hyperparameter Configurations for fine-tuned encoder models	x

1 Introduction

Social media has become one of the main channels through which investors and analysts share their views on markets. On Twitter in particular, thousands of posts about stocks, sectors, and economic events are published every day, making it a relevant source for real-time market sentiment. Automatically classifying this sentiment can support trading signals, risk monitoring, and market analysis tools. The challenge is that financial tweets are short, noisy, and highly domain-specific: they mix standard English with stock tickers, hashtags, and references to earnings events that generic NLP tools are not designed to handle. As a concrete illustration, VADER [1], a widely used lexicon-based sentiment tool, achieves only approximately 48% accuracy on this corpus, which is 16 percentage points below the majority-class baseline of 65%, confirming that off-the-shelf approaches are insufficient for this domain.

This report describes our solution to the three-class sentiment classification task proposed in the Text Mining course project. Given a set of financial tweets, the goal is to predict whether each tweet expresses a Bearish (0), Bullish (1), or Neutral (2) market attitude. The training corpus contains 9,543 labeled tweets, and predictions are required for 2,388 unlabeled test tweets. Since the Neutral class is the most frequent, macro-averaged F1-score (F1-Macro) is used as the primary metric, as it evaluates performance equally across all three classes.

The main contributions of this work can be summarized as follows:

- A domain-aware preprocessing pipeline that maps financial entities such as tickers, percentage movements, monetary values, and quarterly references to dedicated sentinel tokens, preserving their sentiment signal rather than removing them.
- A systematic comparison of 14 feature representations, covering Bag-of-Words, TF-IDF, Word2Vec (CBOW and Skip-Gram), GloVe (Twitter-pretrained), and frozen CLS embeddings from three transformer encoders (FinBERT, DistilBERT, Twitter-RoBERTa), evaluated across eight classical classifiers and two stacked recurrent architectures.
- Fine-tuning of three domain-adapted transformer encoders: FinBERT, Twitter-RoBERTa, and DeBERTa-v3, with the latter achieving the best validation F1-Macro across all models evaluated.
- As extra work: zero-shot and few-shot classification with the Phi-3-mini-4k-instruct decoder model.
- As extra work: a LangChain ReAct agent with a conversational interface that routes tweets across models, compares classifier outputs, and automates evaluation.

The remainder of this report is organized as follows. Section 2 presents the exploratory analysis of the corpus. Section 3 covers the preprocessing pipeline and corpus split. Section 4 describes the feature engineering methods. Section 5 presents the classification models. Section 6 compares all models and justifies the final model choice. Section 7 concludes the report.

2 Data Exploration

This section presents an exploratory analysis of the training corpus, with the goal of understanding the data characteristics that shape the preprocessing and modelling decisions described in subsequent sections. The dataset contains 9,543 labeled tweets distributed across three sentiment classes: Bearish (1,442 tweets, 15.1%), Bullish (1,923 tweets, 20.2%), and Neutral (6,178 tweets, 64.7%). The pronounced dominance of the Neutral class is the most consequential structural property of this corpus: a classifier that always predicts Neutral would already achieve 64.7% accuracy, making raw accuracy a misleading metric and reinforcing the choice of F1-Macro as the primary evaluation criterion. The following subsections examine class distribution, tweet length, vocabulary composition, and domain-specific linguistic patterns.

2.1 Dataset Overview and Class Distribution

The training set contains 9,543 tweets labeled across three sentiment classes, as detailed in Table 1 and Figure 1. The distribution is heavily skewed toward Neutral, which is 4.3 times more frequent than Bearish and 3.2 times more frequent than Bullish. The test set contains 2,388 tweets with no labels, for which predictions are submitted as a deliverable.

2.2 Tweet Length Distribution

As shown in Figure 2, tweets are short documents with a right-skewed word count distribution (standard deviation of 4.7 words). More relevant for modelling is the per-class breakdown: Bearish, Bullish, and Neutral tweets have nearly identical length profiles (means of 12.0, 11.9, and 12.3 words; standard deviations of 4.3, 4.3, and 4.8 respectively), confirming that length is not a discriminative feature for this task and that classification must rely entirely on lexical and semantic content.

2.3 Vocabulary and Word Clouds

As shown in Figure 3, *stock* is the dominant term across all three classes and carries little discriminative power on its own. The classes diverge in the surrounding vocabulary: Bearish tweets are characterised by negative macroeconomic references (*china*, *coronavirus*, *oil*) and results below expectations (*misses*); Bullish tweets by earnings beats and upward revisions (*beats*, *target*, *price*); and Neutral tweets by factual reporting language (*results*, *earnings*, *dividend*, *marketscreener*). Notably, *beats* and *misses* emerge as near-exclusive class markers for Bullish and Bearish respectively, suggesting that earnings surprise language is among the strongest signals available to any classifier trained on this corpus.

2.4 Linguistic and Domain-Specific Observations

Approximately 47% of tweets contain at least one URL, making them the most prevalent noise element in the corpus. Cashtags (\$AAPL, \$TSLA) appear in around 15% of tweets, covering 1,208 unique tickers. Although each ticker appears rarely, their presence signals that the tweet is discussing a specific stock, so we normalize them to a <TICKER> sentinel token rather than discard them entirely. Hashtags are present in 9.4% of tweets and are heavily concentrated in Neutral

content, largely driven by `#marketscreener`, reinforcing the automated-news pattern observed in Section 2.3.

Emojis are essentially absent (0.5% of tweets), and most occurrences are broken encoding artefacts or decorative symbols rather than sentiment-bearing icons, so no dedicated demojization step is needed. Mojibake encoding issues affect only 17 tweets (0.18%) and are handled as a side-effect of standard regex cleaning. Language detection via `langdetect` flags 7.2% of tweets as non-English, but manual inspection shows systematic misclassification: short tweets containing cashtags and financial jargon are consistently assigned incorrect language codes. We therefore apply no language-based filtering.

The VADER [1] baseline makes the domain-specificity problem concrete. As noted in Section 1, VADER scores 48.3% accuracy (Figure 4), below the majority-class baseline. The failure is structural: terms such as *misses*, *downgrade*, and *beats estimates* carry strong directional meaning in financial discourse that VADER’s general-purpose lexicon does not encode. Neutral tweets are the main casualty, with 1,138 misclassified as Bearish and 1,959 as Bullish. This confirms that domain-adapted representations are necessary, and motivates both the preprocessing choices in Section 3 and the encoder selection in Section 4.

2.5 Key Findings

The exploratory analysis reveals five properties of the corpus that directly shape the modelling decisions in subsequent sections:

- **Severe class imbalance.** Neutral accounts for 64.7% of training tweets. A majority-class predictor already achieves 64.7% accuracy, making raw accuracy an unreliable metric and motivating F1-Macro as the primary evaluation criterion throughout this work.
- **Uniformly short tweets.** The median tweet length is 11 raw words (9 after preprocessing), with nearly identical distributions across the three classes. Length alone carries no discriminative signal.
- **Sharp vocabulary differences at the extremes.** *Beats* and *misses* function as near-exclusive class markers for Bullish and Bearish respectively. Neutral content is disproportionately driven by automated aggregator posts (`#marketscreener`), giving it a qualitatively different vocabulary from organic opinion.
- **Sentiment-bearing domain tokens.** Cashtags, percentage movements, and monetary values are prevalent and carry implicit directional signal. Removing them would discard useful features, motivating the sentinel token strategy described in Section 3.
- **Failure of general-purpose sentiment tools.** VADER achieves 48.3% accuracy, 16 percentage points below the majority-class baseline. Financial terms such as *misses* and *downgrade* carry domain-specific directional meaning that a general-purpose lexicon cannot encode, confirming that domain-adapted representations are a prerequisite for competitive performance.

3 Data Preprocessing

This section covers the corpus split procedure (Section 3.1) and the full preprocessing pipeline, which comprises text cleaning and normalisation (Section 3.2), tokenisation and stop word removal (Section 3.3), and lemmatisation and stemming (Section 3.4). A step-by-step trace of the pipeline on representative tweets is provided in Appendix B.

3.1 Corpus Split

The 9,543 labeled tweets were divided into a training set of 7,634 samples and a validation set of 1,909 samples using a stratified 80/20 split. Stratification is necessary given the class imbalance described in Section 2: a random split would not guarantee that the minority classes are proportionally represented in both subsets. The resulting class proportions are identical across train and validation - 64.7% Neutral, 20.2% Bullish, and 15.1% Bearish - ensuring that validation metrics reflect the same distribution the model will face on the test set. The split is applied to raw text before any preprocessing step, preventing validation vocabulary from leaking into the cleaning logic.

3.2 Text Cleaning and Normalisation

All preprocessing is implemented as a single `clean_text()` function with configurable flags, applied sequentially in a fixed order. The pipeline begins with encoding repair via `ftfy`, which resolves the mojibake artefacts identified in Section 2. Emojis are then stripped via `emoji.replace_emoji`, followed by URL removal and the deletion of user mentions (`@user`).

The most consequential step is the replacement of domain-specific entities with sentinel tokens rather than discarding them. The EDA showed that these tokens carry signal: cashtags identify the stock being discussed, percentage movements encode direction, and hashtag density is markedly higher in Neutral content. Six sentinel types are applied: `<ticker>` for cashtags, `<hashtag>` for hashtags, `<pct_up>` and `<pct_down>` for signed percentage movements, `<pct>` for unsigned percentages, `<money>` for monetary values, `<quarter>` for quarterly references, and `<num>` for standalone numeric strings. Major financial indices (S&P 500, Dow Jones, NASDAQ) are normalised to single tokens (`sp500`, `dowjones`, `nasdaq`) so they survive the subsequent alphabetic filter intact.

Contractions are then expanded via the `contractions` library (*don't* → *do not*) before lowercasing, ensuring that negation words are preserved as full tokens for the stop word removal step. Text is then lowercased, residual punctuation is stripped, and sequences of three or more repeated characters are collapsed to two.

3.3 Tokenisation and Stop Word Removal

Tokenisation is performed by whitespace splitting rather than a subword tokeniser, which preserves the multi-character sentinel tokens introduced in Section 3.2 as single units. Stop words are removed using the NLTK English list with two targeted exclusions. Negations (*no*, *not*, *nor*, *never*, *n't*, *none*, *nothing*, *neither*) and market-movement terms (*up*, *down*, *rise*, *fall*, *gain*, *loss*, *above*, *below*, *over*, *under*, and their inflected forms) are retained, as both groups carry directional

signal relevant to distinguishing Bearish from Bullish content. All sentinel tokens are also exempt from stop word removal.

3.4 Lemmatisation and Stemming

Two morphological normalisation variants are produced for all splits. Lemmatisation uses NLTK’s `WordNetLemmatizer`, which maps inflected forms to their dictionary root while preserving readability. Stemming uses the Snowball stemmer (English variant), which applies more conservative truncation rules than the original Porter stemmer and avoids over-aggressive suffix removal on financial vocabulary. Both reduce the training vocabulary substantially: from 27,067 unique tokens in the raw corpus to 11,127 after lemmatisation and 9,423 after stemming.

All split variants - raw text, lemmatised, and stemmed, for train, validation, and test - are serialised together in `processed_splits.pkl`. Raw text is preserved alongside the cleaned variants because transformer models require their own tokenisation and should not receive pre-normalised input. This produces three text variants reusable by all downstream notebooks without rerunning the pipeline.

4 Feature Engineering

Three families of representations are produced. Sparse count-based vectors (Bag-of-Words and TF-IDF) serve as interpretable baselines: they are fast to compute, well-suited to linear classifiers, and give a clear picture of how far lexical features alone can take the classification. Dense static embeddings (Word2Vec and GloVe) encode semantic relationships between words in a continuous space, reducing dimensionality and capturing similarity that sparse vectors miss. Contextual CLS embeddings from pretrained transformer encoders go further by representing each tweet as a function of its full context, with models pre-trained on financial or social media text providing domain-relevant inductive bias from the start.

4.1 Bag-of-Words and TF-IDF

Bag-of-Words represents each tweet as a vector over the training vocabulary, produced in two variants: binary (word presence) and count (word frequency). Both share the same vocabulary, with 11,113 dimensions on the lemmatised corpus and 9,420 on the stemmed corpus.

TF-IDF weights each term by its within-document frequency relative to how rare it is across the corpus, suppressing terms that are either too frequent or too sparse to be informative. A minimum document frequency of 2 (terms that appear in only one document) and a maximum of 90% (terms appearing in >90% of tweets) are applied, yielding vocabularies of 5,076 (lemmatised) and 4,501 (stemmed) for unigrams. A bigram variant extends the vocabulary to spans of one or two tokens, growing to 10,468 and 10,151 dimensions respectively; bigrams capture sentiment-bearing phrases such as *price target* and *hedge fund* that unigrams split into separate tokens. These eight sparse representations form the interpretable baseline group evaluated in Section 5.

4.2 Word Embeddings: Word2Vec and GloVe

Four Word2Vec [2] models are trained on the training corpus: CBOW and Skip-Gram architectures applied to both the lemmatised and stemmed variants. All four share the same hyperparameters:

100 dimensions, window size 5, and minimum token count 2. Tweet-level representations are obtained by mean-pooling the word vectors of all in-vocabulary tokens; tweets where all tokens are out-of-vocabulary receive a zero vector. Training Word2Vec from scratch on approximately 7,600 tweets is a known limitation: the corpus is too small to learn reliable semantic relationships, and the resulting embeddings are expected to underperform richer pretrained alternatives.

To address this, the Twitter-GloVe-100 [3] pretrained embeddings are included as a complementary static representation. These were trained on approximately two billion tweets and cover a vocabulary of 1,193,514 words, providing substantially richer coverage than the in-domain Word2Vec models. The same mean-pooling procedure is applied. All six dense representations have dimensionality 100. In addition to the pooled document vectors, sequential embeddings of shape $(n, 28, 100)$ are saved for use by the recurrent architectures in Section 5.1, where the maximum sequence length of 28 covers all training tweets without truncation.

Figure 5 (Appendix B) compares the three main dense representations, and Figure 6 shows the effect of lemmatisation versus stemming across the Word2Vec variants. All cases exhibit substantial class overlap, with CBOW collapsing into a narrow spike that reflects its training objective. The lack of visible separation anticipates the classification results in Section 5, where dense static embeddings consistently underperform sparse representations.

4.3 Transformer Encoder Embeddings

The primary transformer encoder is FinBERT (`ProsusAI/finbert`, [4]), a BERT-base model further pre-trained on a 4.9 billion-token financial corpus comprising Reuters news, the Financial PhraseBank, and analyst reports. For each tweet, the hidden state of the [CLS] token [5] at the final layer is extracted as a 768-dimensional document embedding. Raw tweet text is used as input, bypassing the lemmatised and stemmed variants, since the model’s own WordPiece tokeniser is designed for unmodified natural language. Embeddings are computed once in batches of 32 with gradients disabled and cached to disk, making them reusable across all downstream classifiers without repeated forward passes.

4.4 Extra Work – Additional Transformer Encoder Methods

This subsection reports extra work beyond the core requirements, targeting the Feature Engineering bonus (0.50 pts per additional Transformer Encoder method, maximum 2 extra methods).

Two additional frozen encoders are evaluated. DistilBERT (`distilbert-base-uncased`, [6]) is a distilled version of BERT-base with 6 transformer layers instead of 12, 40% fewer parameters, and approximately 60% faster inference while retaining around 97% of BERT’s NLU performance. It serves as a lightweight general-purpose reference point, isolating the contribution of domain-specific pre-training when compared against FinBERT. Twitter-RoBERTa (`cardiffnlp/twitter-roberta-base-sentiment-latest`, [7]) is a RoBERTa-base model [8] trained on approximately 124 million tweets from 2018–2021 and subsequently fine-tuned for three-class sentiment classification, making it directly aligned with both the data source and the target task. Both encoders produce 768-dimensional CLS embeddings extracted with the same procedure as FinBERT: raw tweet text, batch size 32, no gradient computation, and cached to disk.

5 Classification Models

Two families of models are evaluated, both required by the project criteria. The first covers traditional and deep machine learning: eight classical classifier families in sixteen configurations, each tested across all relevant feature representations from Section 4, and fourteen recurrent architectures covering single-layer variants (RNN, BiRNN, LSTM) and stacked 2-layer/3-layer variants (RNN, BiRNN, LSTM, BiLSTM), all trained on GloVe (lemmatised) sequences. The second covers three transformer encoders fine-tuned end-to-end on the training set. An additional decoder model experiment is reported as extra work.

5.1 Traditional and Deep ML Models

Eight classical classifier families are evaluated in sixteen configurations, each tested across all 14 feature representations from Section 4, with the exception of Naive Bayes which is restricted to BoW variants due to its non-negativity requirement. Class imbalance is addressed through balanced class weights wherever supported, and through sample weights for XGBoost. Two patterns emerge consistently across all experiments. First, sparse representations outperform dense embeddings across every classical ML family: the mean F1-Macro of sparse configurations (0.709) is approximately 18 percentage points above the mean of dense configurations (0.525), and for every classical classifier family, the best sparse feature outperforms the best dense feature. Second, stemming consistently outperforms lemmatisation on sparse features across the top-tier models, as it reduces the vocabulary more aggressively and concentrates discriminative signal in terms such as *beats* and *misses*.

Recurrent architectures are trained exclusively on GloVe (lemmatised) sequences, confirmed as the strongest dense embedding across every architecture tested. Six single-layer architectures (RNN, BiRNN, and LSTM in two unit configurations each) are evaluated in the shallow section, and eight stacked variants (2-layer and 3-layer RNN, BiRNN, LSTM, and BiLSTM) in the deep section. The best recurrent model is BiLSTM (3 layers: 128→64→32) at F1-Macro 0.651, approximately 11 percentage points below the best classical model. When comparing the best result per model family, every classical ML family outperforms the best recurrent architecture, confirming that sequential models cannot compensate for the information contained in sparse lexical representations at this corpus size. Table 4 (Appendix D) reports the best-performing configuration per model family.

5.2 Frozen Encoder Embeddings as Features

The CLS embeddings extracted in Section 4.3 and Section 4.4 are used here as fixed-length feature vectors fed into three classifiers: Logistic Regression, LinearSVC, and XGBoost. This produces nine encoder–classifier combinations, evaluated on the same validation set as Section 5.1. Table 5 reports the best-performing classifier per encoder.

XGBoost is the best classifier for the two domain-adapted encoders but not for DistilBERT, where LinearSVC leads. This suggests that XGBoost’s non-linear boundaries add value when the embedding space has sufficient structure, but offer no advantage over a linear separator when the representation is weaker. The encoder ordering – Twitter-RoBERTa, FinBERT, DistilBERT – holds across every classifier, confirming that domain and style alignment with the target corpus

matter more than raw model size. The best combination, XGBoost on Twitter-RoBERTa CLS vectors (F1 0.796), surpasses the best sparse-feature classical model from Section 5.1 (LinearSVC, F1 0.760) by 3.6 percentage points, demonstrating that contextual representations capture signal that TF-IDF cannot. Whether updating the encoder weights end-to-end can improve further is addressed in Section 5.3.

5.3 Fine-tuned Transformer Encoders

Unlike Section 4.4, where encoder weights were kept frozen and only a downstream classifier was trained, here the full model is updated end-to-end. A linear classification head is placed on top of the [CLS] representation and the encoder and head are optimised jointly on the training set using AdamW with weighted cross-entropy loss. The complete training configuration is reported in Appendix E; the three most consequential choices are layer-wise learning rate decay (LLRD, $\gamma = 0.9$), which applies progressively lower learning rates to earlier encoder layers to protect the pretrained representations from large gradient updates; a 10% linear warmup followed by linear decay; and early stopping with patience 5 on validation F1-Macro, run on a T4 GPU.

The two models carried forward from Section 4.4 are FinBERT (ProsusAI/finbert, [4]) and Twitter-RoBERTa (cardiffnlp/twitter-roberta-base-sentiment-latest, [7]). In the frozen-embedding setting, XGBoost on Twitter-RoBERTa CLS vectors achieved the highest F1-Macro of that section (0.796), and XGBoost on FinBERT followed closely (0.777); the two together accounted for the top five results in the frozen encoder ranking. DistilBERT ranked last among the three frozen encoders (F1 0.719, roughly 5 points below FinBERT) and was not included. A third model, DeBERTa-v3 (nickmuchi/deberta-v3-base-finetuned-finance-text-classification, [9]), was identified after the frozen-embedding experiments were already complete; given the additional GPU cost of extracting and caching a new set of embeddings at that stage, it was evaluated directly in the fine-tuning setting instead. DeBERTa-v3 [9] uses disentangled attention and ELECTRA-style pre-training, encoding token content and relative position in separate matrices rather than summing them, and consistently outperforms RoBERTa-base on NLU benchmarks. Its checkpoint also starts from a DeBERTa-v3-base already fine-tuned on financial news classification, providing two stages of domain adaptation before tweet-level training. Because it is more sensitive to large gradient updates than RoBERTa-family models, the learning rate is lowered to 1×10^{-5} (versus 2×10^{-5} for the other two) and the batch size is reduced to 16 to stay within T4 memory.

Table 6 reports the validation results. DeBERTa-v3 reaches the highest F1-Macro and accuracy of any model in this project. Fine-tuning Twitter-RoBERTa improves its F1 from 0.796 to 0.854, a gain of 5.8 percentage points over its best frozen-embedding configuration; FinBERT improves from 0.777 to 0.820, a gain of 4.3 points. The larger gain for Twitter-RoBERTa is consistent with a stronger alignment between its Twitter-domain pre-training and tweet-level adaptation. The comparison with the full model ranking is discussed in Section 6.

5.4 Extra Work – Decoder Model for Classification

This subsection reports extra work beyond the core requirements, targeting the Classification Models bonus (1.00 pt for correctly using a decoder model for classification).

Unlike the encoder models in Sections 4.3, 5.2 and 5.3, which represent a tweet as a fixed vector and pass it to a classification head, this approach instructs a decoder-only language model to generate a label word directly. The model used is `microsoft/Phi-3-mini-4k-instruct` [10], a 3.8B-parameter instruction-following decoder loaded in `bfloat16` with automatic device mapping to manage GPU memory. No fine-tuning is performed; the model weights remain fixed throughout.

Two prompting strategies are evaluated. In zero-shot mode, the model receives a structured system message followed by the tweet text. The message is built from six components, detailed in Appendix C.

The first token of the generated output is extracted and mapped to the project label set. Greedy decoding (`do_sample=False`) is used throughout, making results fully deterministic. In few-shot mode, nine labelled examples (three per class) are embedded as alternating chat turns before the tweet to classify, keeping the same system message so that the only variable relative to zero-shot is the presence of examples. The few-shot prompt structure is shown in Appendix C.

Zero-shot achieves F1-Macro 0.719, but with a systematic failure mode: Bearish and Bullish recall are high (0.85 and 0.92) (Figure 7) while their precision is low (0.62 and 0.54), meaning the model aggressively assigns non-Neutral labels to tweets that are in fact Neutral. Without examples to calibrate the class boundary, the model defaults to expressing opinion even on factual content. Few-shot resolves this: nine examples raise F1-Macro to 0.798, a gain of 7.9 percentage points. Precision for Bearish and Bullish climbs to 0.72 and 0.82 respectively, and Neutral recall recovers from 0.64 to 0.88. The few-shot result marginally exceeds the best frozen encoder (XGBoost + Twitter-RoBERTa, F1 0.796) and sits 7.3 percentage points below the best fine-tuned model (DeBERTa-v3, F1 0.871), despite using no labelled training data whatsoever.

6 Evaluation and Results

6.1 Overall Model Comparison

Table 7 (Appendix D) consolidates the validation performance of all models evaluated across sections, ordered by F1-Macro within each group. F1-Macro is the primary metric throughout: with Neutral accounting for 64.7% of the validation set, raw accuracy rewards predicting the majority class regardless of whether Bearish or Bullish content is detected at all. Macro-averaging assigns equal weight to all three classes and directly penalises that failure mode. Accuracy, macro Precision, and macro Recall are also reported per the project rubric and allow cross-checking of the aggregate F1 scores.

Four regimes are visible across the groups. The VADER lexicon sits at the bottom, confirming that general-purpose sentiment tools are structurally insufficient for financial text. Classical sparse-feature models cluster between F1 0.676 and 0.760, with LinearSVC at the top. Recurrent architectures on GloVe peak at BiLSTM 3L (F1 0.651), roughly 11 points below the classical ceiling, illustrating that sequential capacity cannot compensate for the discriminative information already captured by simple lexical statistics. Frozen transformer encoder embeddings break through the classical ceiling: XGBoost on Twitter-RoBERTa CLS vectors reaches F1 0.796, 3.6 points above LinearSVC. Fine-tuned encoders shift to a substantially higher tier, with DeBERTa-

v3 at F1 0.871 representing an 11-point gain (Figure 8) over the best classical model. Phi-3-mini few-shot, using no labelled training data, places between the frozen and fine-tuned tiers at F1 0.798.

6.2 Per-Class Analysis

From Figure 9, the dominant error pattern for DeBERTa-v3 is absorption of minority-class tweets into Neutral rather than cross-class confusion between Bearish and Bullish. Bearish–Bullish swaps are rare, which is the right failure mode to minimise: a system that misclassifies a Bearish tweet as Neutral misses a sell signal and incurs an opportunity cost, but does not actively mislead. Misclassifying Bearish as Bullish inverts the directional signal entirely, potentially prompting a buy-side decision on genuinely negative news – a substantially more damaging error in any applied trading context. The same asymmetry holds in the other direction: Bullish predicted as Neutral is a missed buy signal, while Bullish predicted as Bearish would trigger a short on a positive event.

DeBERTa-v3 handles minority classes with considerable balance. Bearish precision and recall are 0.829 and 0.844, Bullish 0.839 and 0.868, and Neutral 0.931 and 0.917 (Table 8). The gap between Neutral and minority-class recall – 7–8 percentage points – is narrow compared to classical models, where the same gap often exceeded 25 points. The small gap between Bearish precision/recall (0.829/0.844) and Bullish precision/recall (0.839/0.868) confirms that DeBERTa-v3 generalises comparably across both minority classes despite Bullish having a third more training examples (1,923 vs 1,442) – the model is not simply benefiting from higher support in one class.

6.3 Final Model Selection

DeBERTa-v3 fine-tuned is selected as the final model on the basis of its validation F1-Macro of 0.871, the highest recorded across all approaches evaluated in this project. The result reflects two structural advantages. First, the starting checkpoint (the model itself) was already fine-tuned on financial news classification before tweet-level training, giving the model financial domain knowledge as a starting point rather than something to be learned from scratch. Second, DeBERTa-v3’s disentangled attention architecture has consistently outperformed RoBERTa-base-level models on natural language understanding benchmarks, and proves more effective at picking up the directional signals in short financial tweets – where a single word such as *beats* or *cuts* can determine the sentiment of the entire message.

For the final submission, the model is retrained on the full 9,543-tweet training corpus without a validation split, using the same hyperparameters established in Section 5.3 (excluding a slight increase in epochs, due to the increase of training instances). Test-set predictions for the 2,388 unlabeled tweets in `test.csv` are stored in `pred_24.csv`.

6.4 Extra Work – Agentic AI Workflow (Extra Challenge 2)

This subsection reports the agentic workflow developed as Extra Challenge 2, targeting up to 1.50 pts. The agent implements a conversational interface and performs non-trivial decision and coordination tasks, as required by the project handout.

The agent is built on LangChain’s ReAct framework [11] with Azure OpenAI’s ChatGPT as the

reasoning backbone. A `ConversationBufferMemory` instance maintains the full dialogue across turns, allowing the agent to reference earlier classifications or evaluation results when answering follow-up questions without the user repeating context.

Four tools are registered. Three are single-tweet classifiers – `classify_with_deberta`, `classify_with_roberta`, and `classify_with_finbert` – each running one forward pass through the corresponding fine-tuned model and returning the predicted label alongside per-class probabilities. The fourth, `evaluate_batch`, accepts a path to a labelled CSV file and runs all three models across the full dataset, reporting per-model accuracy, macro F1, per-class precision and recall, and the inter-model agreement rate.

The non-trivial coordination is specified in the agent’s system prompt rather than hardcoded in Python, so the LLM must reason through each decision at inference time. The protocol has three tiers.

Tier 1 – Multi-model routing. The agent always calls DeBERTa-v3 first, then Twitter-RoBERTa. If both agree, the shared label is returned with HIGH confidence and FinBERT is never invoked. This routing is non-trivial because the agent decides whether to call a third tool based on the outputs of the first two – it is not a fixed chain but a conditional sequence driven by intermediate observations.

Tier 2 – Majority-vote tiebreaking. When DeBERTa-v3 and Twitter-RoBERTa disagree, the agent calls FinBERT as a tiebreaker and returns the majority label with MEDIUM confidence. The decision of when to invoke FinBERT – and that it should only be invoked in this specific situation – is left to the LLM’s own reasoning rather than a branching script.

Tier 3 – LLM-as-a-judge. When all three models disagree and no majority exists, the agent is instructed to stop calling tools entirely. Instead, it reads the tweet text directly, identifies directional language and financial terminology, and issues a final label as LLM-judge with an explicit reasoning trace. This mechanism is defined in the system prompt and the architecture supports it, but the demo conversations in the notebook did not produce a three-way disagreement with no majority – the most ambiguous case encountered, the tweet “Fed holds rates steady”, resolved as a Tier 2 majority vote with two models predicting Bullish. Tier 3 nonetheless qualifies the system as genuinely agentic: it replaces tool use with the model’s own financial knowledge as a final arbiter, and the ReAct trace provides a full audit trail for any such decision.

Beyond individual tweet classification, the `evaluate_batch` tool enables a single natural-language prompt to trigger multi-model evaluation over a full labelled dataset. On a 100-tweet validation sample, the three-model ensemble achieved accuracy 0.930 and F1-Macro 0.914, above any individual model, with inter-model agreement on 82% of tweets. The agent’s conversational memory also allows derived questions – such as which model to trust as a tiebreaker given the evaluation results – to be answered in the same session without re-running the evaluation.

7 Conclusions

This work developed a full NLP pipeline for three-class market sentiment classification of financial tweets, covering corpus analysis, domain-aware preprocessing, systematic feature and model evaluation, and agentic deployment. The central methodological contribution is a preprocessing strategy based on sentinel token substitution, which preserves the discriminative signal carried by cashtags, percentage movements, monetary values, and quarterly references rather than discarding them as noise. On top of this foundation, 8 classical classifier families were evaluated across 14 feature representations, 8 recurrent architectures across GloVe embeddings, 3 frozen transformer encoders paired with downstream classifiers, and 3 fine-tuned encoder models. DeBERTa-v3, starting from a checkpoint pre-adapted to financial news, achieves the highest validation F1-Macro of 0.871, representing an 11-point gain over the best sparse-feature classical model. Two extra components extend the pipeline: Phi-3-mini zero-shot and few-shot classification demonstrated that a well-prompted decoder with nine hand-written examples reaches F1 0.798 with no labelled training data, and the LangChain ReAct agent implements a three-tier routing protocol with a conversational interface and an LLM-as-a-judge fallback for cases where no model majority exists.

Three limitations are worth acknowledging. Class imbalance is structural: Neutral accounts for 64.7% of training tweets, and even with weighted loss functions the minority classes receive proportionally less training signal throughout. Out-of-vocabulary tickers are collapsed into a single `<ticker>` sentinel, discarding company identity that a named-entity-aware encoder could leverage for stock-specific sentiment. The decoder experiment is constrained by GPU memory: Phi-3-mini is run inference-only, so the gap between few-shot prompting and fine-tuned encoders (roughly 7 percentage points) reflects the cost of not updating any weights.

Future work can address each limitation concretely. Replacing the weighted cross-entropy loss with focal loss or applying targeted oversampling on hard Bearish examples may narrow the minority-class gap without increasing training time. Dynamic few-shot retrieval – selecting the most semantically similar labelled examples per tweet at inference time rather than using fixed hand-written examples – would likely push the decoder ceiling closer to the fine-tuned baseline with no additional annotation cost. Finally, the Twitter-RoBERTa pre-training window covers 2018–2021, and financial Twitter vocabulary shifts with market cycles; continual adaptation on more recent tweet streams would be necessary for a production deployment.

References

- [1] Clayton J. Hutto and Eric Gilbert. “VADER: A Parsimonious Rule-Based Model for Sentiment Analysis of Social Media Text”. In: *Proceedings of the International AAAI Conference on Web and Social Media* (2014).
- [2] Tomas Mikolov et al. “Distributed Representations of Words and Phrases and their Compositionality”. In: *Advances in Neural Information Processing Systems* 26 (2013).
- [3] Jeffrey Pennington, Richard Socher, and Christopher D. Manning. “GloVe: Global Vectors for Word Representation”. In: *Proceedings of the 2014 Conference on Empirical Methods in Natural Language Processing (EMNLP)*. 2014.
- [4] Dogu Araci. “FinBERT: Financial Sentiment Analysis with Pre-trained Language Models”. In: *arXiv preprint arXiv:1908.10063* (2019).
- [5] Jacob Devlin et al. “BERT: Pre-training of Deep Bidirectional Transformers for Language Understanding”. In: *arXiv preprint arXiv:1810.04805* (2019).
- [6] Victor Sanh et al. “DistilBERT, a distilled version of BERT: smaller, faster, cheaper and lighter”. In: *arXiv preprint arXiv:1910.01108* (2019).
- [7] Daniel Loureiro et al. “TimeLMs: Diachronic Language Models from Twitter”. In: *Proceedings of the 60th Annual Meeting of the Association for Computational Linguistics: System Demonstrations*. 2022, pp. 251–260. DOI: [10.18653/v1/2022.acl-demo.25](https://doi.org/10.18653/v1/2022.acl-demo.25).
- [8] Yinhan Liu et al. “RoBERTa: A Robustly Optimized BERT Pretraining Approach”. In: 2019.
- [9] Pengcheng He, Jianfeng Gao, and Weizhu Chen. “DeBERTaV3: Improving DeBERTa using ELECTRA-Style Pre-Training with Gradient-Disentangled Embedding Sharing”. In: *arXiv preprint arXiv:2111.09543* (2021).
- [10] Marah Abdin et al. “Phi-3 Technical Report: A Highly Capable Language Model Locally on Your Phone”. In: *arXiv preprint arXiv:2404.14219* (2024). DOI: [10.48550/arXiv.2404.14219](https://doi.org/10.48550/arXiv.2404.14219).
- [11] LangChain. *LangChain: Building applications with LLMs through composability*. <https://github.com/langchain-ai/langchain>. 2023.

A EDA Material

Table 1: Class distribution in the training set.

Class	Count	Proportion (%)
Bearish (0)	1,442	15.1
Bullish (1)	1,923	20.2
Neutral (2)	6,178	64.7
Total	9,543	100.0

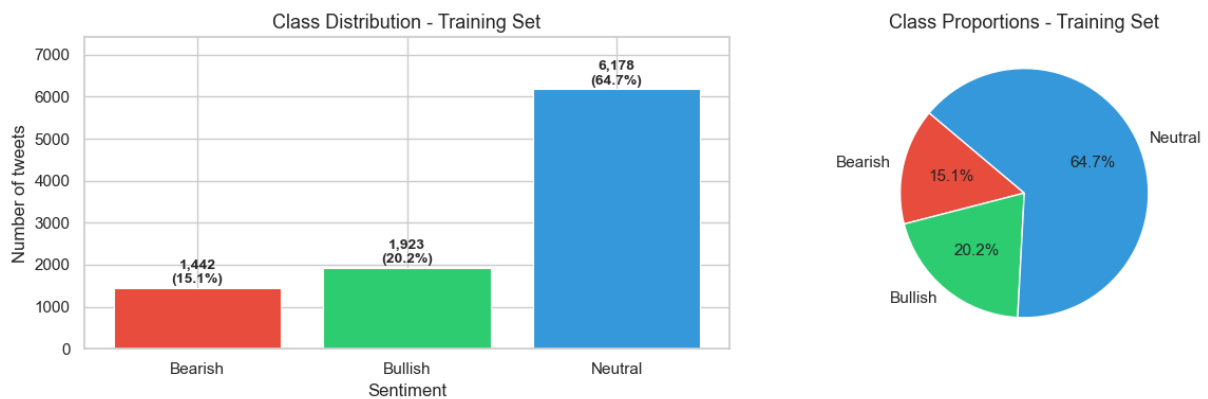


Figure 1: Class distribution of the training set: absolute counts (left) and proportions (right).

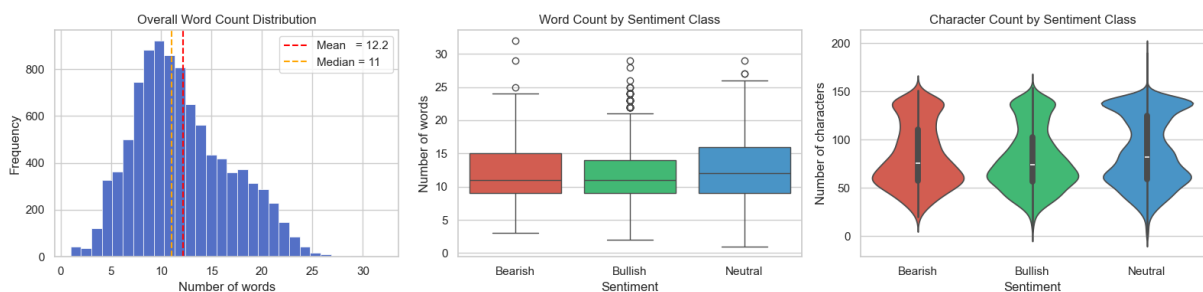


Figure 2: Tweet length distribution: overall word count histogram (left), word count per class (centre), and character count per class (right).



Figure 3: Word clouds per sentiment class (stop words removed).

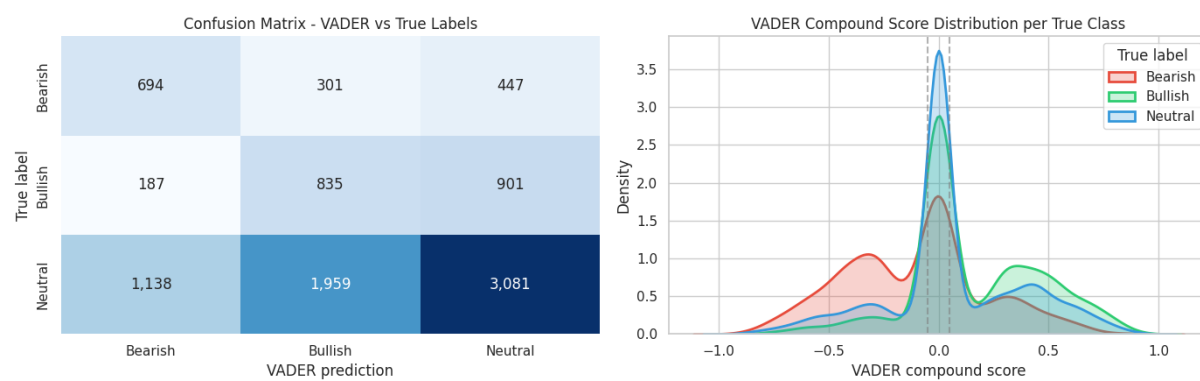


Figure 4: Vader Confusion Matrix (left), Compound Score Distribution per True Class (right)

B Preprocessing Pipeline Summary

Table 2 lists the ten steps of the `clean_text()` pipeline in execution order. Table 3 shows the transformation applied to four representative tweets, taken directly from the notebook sanity check.

Table 2: Preprocessing pipeline steps in execution order.

Step	Technique	Effect
1	Encoding repair	<code>ftfy</code> resolves mojibake and broken Unicode
2	Emoji removal	<code>emoji.replace_emoji</code> strips emoji characters
3	URL & mention removal	Regex removes URLs and <code>@user</code> tokens
4	Sentinel substitution	Cashtags $\rightarrow$ <code><ticker></code> ; hashtags $\rightarrow$ <code><hashtag></code> ; signed percentages $\rightarrow$ <code><pct_up>/<pct_down></code> ; unsigned $\rightarrow$ <code><pct></code> ; monetary values $\rightarrow$ <code><money></code> ; quarterly refs $\rightarrow$ <code><quarter></code> ; numbers $\rightarrow$ <code><num></code> ; major indices normalised to <code>sp500</code> , <code>dowjones</code> , <code>nasdaq</code>
5	Contraction expansion	<code>contractions.fix</code> expands e.g. <i>don't</i> $\rightarrow$ <i>do not</i> before lowercasing
6	Lowercasing	All characters converted to lowercase
7	Punctuation removal	Residual non-alphanumeric characters stripped; sequences of 3+ repeated characters collapsed to 2
8	Tokenisation	Whitespace split (<code>str.split()</code>)
9	Stop word removal	NLTK English list minus negations (<i>no</i> , <i>not</i> , <i>nor</i> , <i>never</i> , <i>n't</i> , <i>none</i> , <i>nothing</i> , <i>neither</i>) and market-movement terms (<i>up</i> , <i>down</i> , <i>rise</i> , <i>fall</i> , <i>gain</i> , <i>loss</i> , <i>above</i> , <i>below</i> , <i>over</i> , <i>under</i> and inflected forms); sentinels always retained
10	Lemmatisation/Stemming	WordNet lemmatiser or Snowball stemmer; sentinel tokens kept intact

Table 3: Example transformations for four representative tweets (lemmatised and stemmed variants).

	Text
Input	\$AAPL beats Q3 estimates! Stock jumps +5% to \$175.50 https://t.co/abc @investor #earnings
Lemmatised	<ticker> beat <quarter> estimate stock jump <pct_up> <money> <hashtag>
Stemmed	<ticker> beat <quarter> estim stock jump <pct_up> <money> <hashtag>
Input	Tesla doesn't disappoint, not even close. Stock down 3% on the S&P 500.
Lemmatised	tesla not disappoint not even close stock down <pct> sp500
Stemmed	tesla not disappoint not even close stock down <pct> sp500
Input	3Q2020 results miss; revenue plunges -12.5% to \$1.2B
Lemmatised	<quarter> result miss revenue plunge <pct_down> <money>
Stemmed	<quarter> result miss revenu plung <pct_down> <money>
Input	NASDAQ rises while Dow Jones falls below 30000
Lemmatised	nasdaq rise dowjones fall below <num>
Stemmed	nasdaq rise dowjones fall below <num>

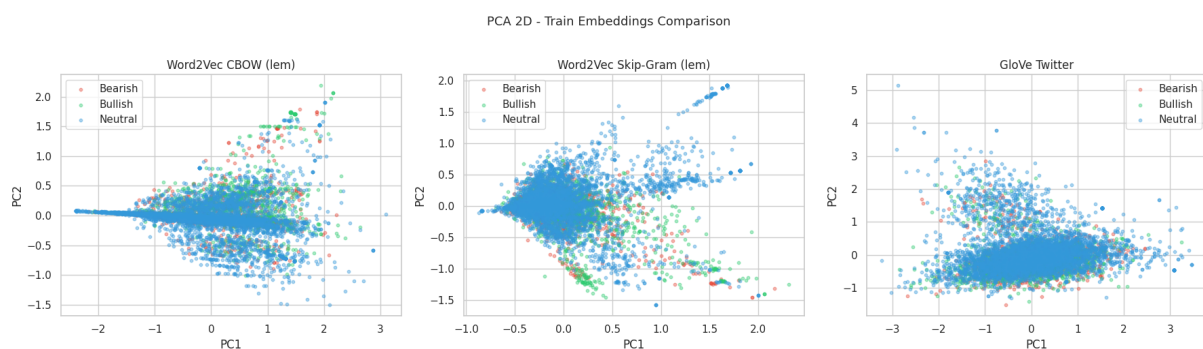


Figure 5: PCA 2D projections of Word2Vec CBOW, Word2Vec Skip-Gram, and GloVe Twitter embeddings (lemmatised). All three show heavy class overlap with no visible linear separation.

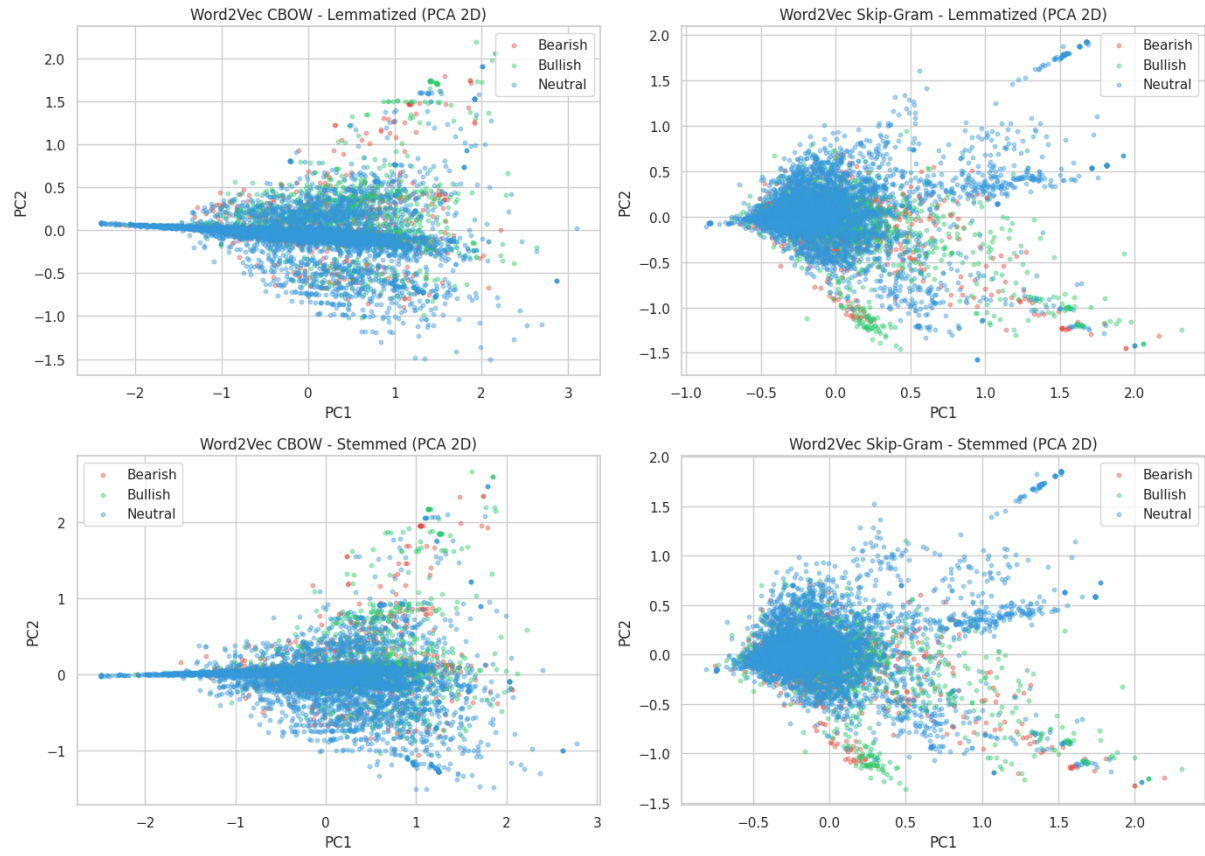


Figure 6: PCA 2D projections of Word2Vec CBOW and Skip-Gram embeddings (lemmatised and stemmed variants). All four cases show heavy class overlap with no visible linear separation.

C Decoder Model Prompts

Zero-shot prompt structure

<i>Persona</i>	You are a financial sentiment analyst.
<i>Instruction</i>	Classify the sentiment of the following market tweet into exactly one of: Bearish, Bullish, or Neutral.
<i>Context</i>	Bearish means the tweet expresses a negative outlook on stocks or markets. Bullish means the tweet expresses a positive or optimistic market outlook. Neutral means the tweet is neither clearly positive nor negative.
<i>Audience</i>	This classification will be used by an automated quantitative trading system.
<i>Tone</i>	Be precise and do not explain your answer.
<i>Format</i>	Respond with a single word only: Bearish, Bullish, or Neutral.

Few-shot prompt structure

<i>System</i>	[same system message as zero-shot]
<i>User</i>	Tweet: Stock markets plunge as recession fears mount amid rising interest rates.
<i>Assistant</i>	Bearish
<i>User</i>	Tweet: Massive sell-off today - investors are fleeing equities for safe-haven assets.
<i>Assistant</i>	Bearish
<i>User</i>	Tweet: Earnings miss sends shares tumbling 12% in after-hours trading.
<i>Assistant</i>	Bearish
[3 Bullish examples] [3 Neutral examples]	
<i>User</i>	Tweet: {tweet to classify}

D Model Results

Table 4: Validation performance of traditional and deep ML models (best configuration per family).

Model	Best Features	Accuracy	Precision	Recall	F1-Macro
KNN (k=15, distance)	TF-IDF Uni (stem)	0.798	0.787	0.647	0.688
Logistic Regression (L2)	TF-IDF Bi (stem)	0.810	0.745	0.758	0.751
Naive Bayes (MultinomialNB)	BoW Count (lem)	0.792	0.724	0.713	0.718
LinearSVC (C=0.1)	BoW Binary (stem)	0.822	0.773	0.749	0.760
Random Forest (200 trees)	BoW Binary (stem)	0.818	0.811	0.692	0.734
XGBoost (lr=0.1, 200 trees)	BoW Binary (stem)	0.811	0.743	0.752	0.748
LightGBM (500 trees, leaves=63)	BoW Binary (stem)	0.791	0.718	0.731	0.724
MLP (512→256)	BoW Count (stem)	0.821	0.783	0.725	0.750
BiLSTM (3L, stacked)	GloVe (lem)	0.713	0.634	0.684	0.651
BiLSTM (2L, stacked)	GloVe (lem)	0.696	0.623	0.689	0.642

Table 5: Validation performance of frozen encoder embeddings with the best downstream classifier per encoder.

Encoder	Best Classifier	Accuracy	Precision	Recall	F1-Macro
DistilBERT	LinearSVC	0.791	0.717	0.722	0.719
FinBERT	XGBoost	0.834	0.789	0.767	0.777
Twitter-RoBERTa	XGBoost	0.846	0.803	0.790	0.796

Table 6: Validation performance of fine-tuned encoder models.

Model	Accuracy	Precision	Recall	F1-Macro
FinBERT (fine-tuned)	0.8628	0.8188	0.8239	0.8198
Twitter-RoBERTa (fine-tuned)	0.8848	0.8460	0.8635	0.8543
DeBERTa-v3 (fine-tuned)	0.8963	0.8665	0.8763	0.8713

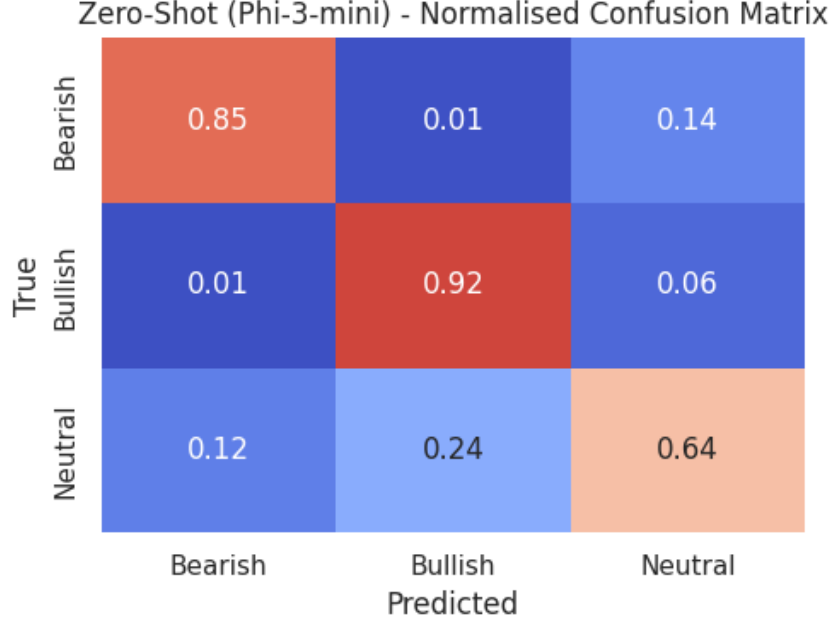


Figure 7: Zero-shot Confusion Matrix

Table 7: Full model comparison on the validation set (ordered by F1-Macro within each group).

Model	Accuracy	Precision	Recall	F1-Macro
VADER baseline (lexicon)	0.483	–	–	0.438
LinearSVC (C=0.1)	0.822	0.773	0.749	0.760
Logistic Regression (L2)	0.810	0.745	0.758	0.751
MLP (512→256)	0.821	0.783	0.725	0.750
XGBoost	0.811	0.743	0.752	0.748
Random Forest	0.818	0.811	0.692	0.734
LightGBM	0.791	0.718	0.731	0.724
Naive Bayes (MultinomialNB)	0.792	0.792	0.713	0.718
KNN (best config)	0.798	0.787	0.647	0.688
BiLSTM (3L stacked)	0.713	0.634	0.684	0.651
Twitter-RoBERTa CLS + XGBoost	0.846	0.803	0.790	0.796
FinBERT CLS + XGBoost	0.834	0.789	0.767	0.777
DistilBERT CLS + LinearSVC	0.791	0.717	0.722	0.719
DeBERTa-v3 (fine-tuned)	0.896	0.867	0.876	0.871
Twitter-RoBERTa (fine-tuned)	0.885	0.846	0.864	0.854
FinBERT (fine-tuned)	0.863	0.819	0.824	0.820
Phi-3-mini few-shot <i>extra</i>	0.837	0.802	0.795	0.798
Phi-3-mini zero-shot <i>extra</i>	0.730	0.695	0.805	0.719

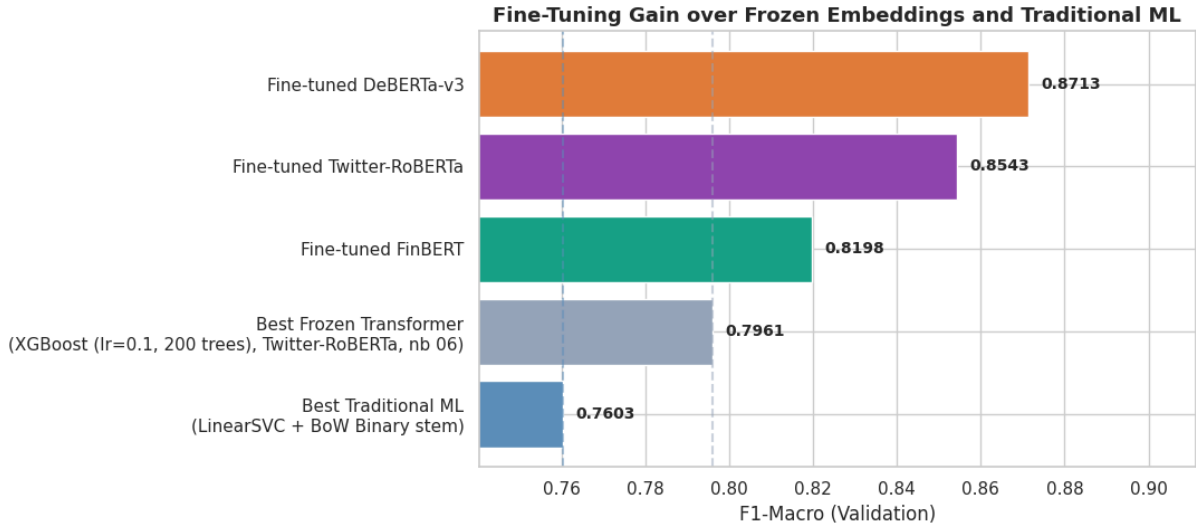


Figure 8: Fine-Tuning Gains

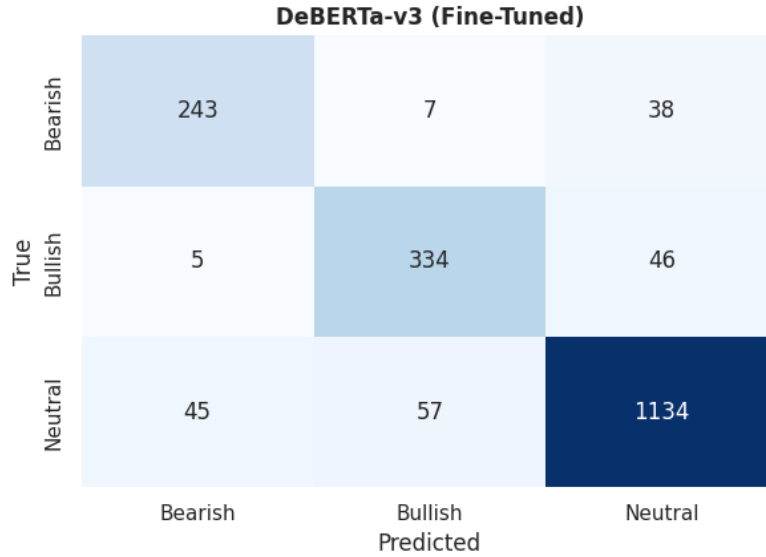


Figure 9: Confusion matrix for the best-performing model (DeBERTa-v3 fine-tuned) on the validation set.

Table 8: Per-class precision and recall for all three fine-tuned encoder models on the validation set.

Model	Acc	F1	P	R	Bearish		Bullish		Neutral	
					P	R	P	R	P	R
DeBERTa-v3 (fine-tuned)	0.896	0.871	0.867	0.876	0.829	0.844	0.839	0.868	0.931	0.917
Twitter-RoBERTa (fine-tuned)	0.885	0.854	0.846	0.864	0.792	0.819	0.814	0.865	0.932	0.906
FinBERT (fine-tuned)	0.863	0.820	0.819	0.824	0.711	0.802	0.835	0.761	0.911	0.909

P and R denote macro Precision and macro Recall in columns 4–5, and per-class Precision and Recall in columns 6–11.

E Hyperparameter Configurations for fine-tuned encoder models

Table 9 reports the training configuration used for all three fine-tuned encoder models. DeBERTa-v3 overrides two shared values due to its sensitivity to large gradient updates.

Table 9: Fine-tuning hyperparameters. DeBERTa-v3 overrides are shown in parentheses where they differ from the shared defaults.

Hyperparameter	Value	Notes
<i>Architecture</i>		
Max sequence length	128 tokens	Covers >99% of tweets after tokenisation
<i>Optimiser (AdamW)</i>		
Learning rate	2e-5 (1e-5 for DeBERTa-v3)	Standard BERT fine-tuning range
Weight decay	0.01	Applied to all params except bias and LayerNorm
LLRD decay γ	0.9	Each lower encoder layer scaled by $\times 0.9$
<i>Scheduler</i>		
Schedule	Linear warmup + linear decay	
Warmup ratio	10%	First 10% of total steps ramp from 0 to peak LR
<i>Training loop</i>		
Batch size	32 (16 for DeBERTa-v3)	Largest fitting T4 GPU (16 GB)
Max epochs	20	Early stopping typically triggers earlier
Early stopping patience	5 epochs	Monitors validation F1-Macro
<i>Loss function (CrossEntropyLoss)</i>		
Label smoothing	0.05	Softens hard targets to reduce overconfidence
Class weight – Bearish	2.205	Computed via <code>compute_class_weight('balanced')</code>
Class weight – Bullish	1.655	
Class weight – Neutral	0.515	